



SISTEMAS OPERATIVOS - LABORATORIO
18 de Diciembre de 2025, Turno 1

Nombre _____ DNI _____
Apellidos _____ Grupo _____

INSTRUCCIONES

- Se debe entregar una carpeta individual con el código de cada apartado (1A, 1B, etc.), para así garantizar la evaluación independiente de cada uno.
- Si el código no compila o su ejecución produce un error grave la puntuación de ese apartado será 0.

Cuestión 1. (5 puntos) Ampliar la funcionalidad del programa `run_commands` (Ejercicio 1 - Práctica 4) de la siguiente forma:

a) **(2,5 puntos)** Implementar una nueva opción `-B`, que --cuando se incluya en la línea de comando-- permita ejecutar algunos de los comandos pasados en el fichero (especificado vía `-s`) sin esperar a su terminación, procediendo al lanzamiento inmediato del siguiente comando. Los comandos del fichero que se tratarán de esta forma serán aquellos que incluyan un `"&"` al final. El programa `run_commands` deberá entonces detectar si el comando acaba con `"&"`, y solo si la opción `-B` está indicada se procederá a su ejecución sin esperar inmediatamente a que termine. Una vez se hayan lanzado a ejecución todos los comandos, `run_commands` esperará a todos los procesos hijos de los comandos que incluyesen `"&"`. Si `-B` no se especifica al lanzar el programa, el terminador `"&"` simplemente se ignorará, solamente eliminándolo del comando si aparece.

Se proponen dos recomendaciones para la implementación:

- Mantener una estructura de datos --por simplicidad, array de longitud máxima prefijada-- que permita almacenar los PIDs de los procesos hijo correspondientes a los comandos acabados en `"&"`.
- Usar la llamada `waitpid()` para esperar a cada proceso hijo creado cuando proceda.

b) **(2,5 puntos)** Implementar una nueva opción `-L` que permita especificar la ruta de un fichero de *log* donde se redirigirán tanto los mensajes que el programa `run_commands` imprime por la salida estándar, como la salida de todos los comandos ejecutados (vía `-x` o `-s`). Para lograr la redirección deseada de los mensajes de `run_commands`, estos se imprimirán al fichero de *log*, que deberá abrir en modo *append* (consultar páginas de manual). Asimismo, el programa modificado utilizará el soporte de redirecciones de `bash` para lanzar el comando o comandos especificados por el usuario. Para ello se aconseja la definición de la siguiente función:

```
pid_t launch_command_redirection(char* cmd, char* logfile_path);
```

Esta llamada, que se utilizará en lugar de `launch_command()` cuando se pase la opción `-L`, realizará las siguientes acciones:

- El programa eliminará del comando `cmd` (string sin tokenizar) los saltos de línea y `'&'` reemplazándolos por espacios.
- Construirá un array `argv` con 3 elementos: (0) `"/bin/bash"`, (1) `"-c"` y (2) `"<cmd> >> <logfile_path>"` (Es decir el comando de redirección que se pasará a `bash`)
- Creará un proceso hijo que ejecute el comando correspondiente al array `argv` anterior
- El padre retornará el `pid` del proceso hijo, sin esperar a su terminación.

Ejemplo de ejecución

```
# Prueba apartado A
$ cat test-bg.txt
sleep 3 &
echo uno
echo dos
sleep 2 &
echo tres
```

```
$ ./run_commands -B -s test-bg.txt
@@ Running command #0: sleep 3 &
@@ Running command #1: echo uno
uno
@@ Command #1 terminated (pid: 4986, status: 0)
@@ Running command #2: echo dos
dos
@@ Command #2 terminated (pid: 4987, status: 0)
@@ Running command #3: sleep 2 &
@@ Running command #4: echo tres
tres
@@ Command #4 terminated (pid: 4989, status: 0)
@@ Command #0 terminated (pid: 4985, status: 0)
@@ Command #3 terminated (pid: 4988, status: 0)

## Prueba apartado B
$ cat test-redir.txt
echo Hello
ls -l
echo Goodbye

$ rm -f log.txt
$ ./run_commands -L log.txt -s test-redir.txt
$ cat log.txt
@@ Running command #0: echo Hello
Hello
@@ Command #0 terminated (pid: 5522, status: 0)
@@ Running command #1: ls -l
total 64
-rwxr-xr-x 1 usuario usuario 267 Oct 22 2024 Makefile
-rw-r--r-- 1 usuario usuario 117 Dec 12 20:30 log.txt
-rwxr-xr-x 1 usuario usuario 22712 Dec 12 20:11 run_commands
-rwxr-xr-x 1 usuario usuario 6895 Dec 12 20:11 run_commands.c
-rw-r--r-- 1 usuario usuario 15328 Dec 12 20:11 run_commands.o
-rw-r--r-- 1 usuario usuario 48 Dec 12 18:35 test-bg.txt
-rw-r--r-- 1 usuario usuario 30 Dec 12 20:29 test-redir.txt
@@ Command #1 terminated (pid: 5523, status: 0)
@@ Running command #2: echo Goodbye
Goodbye
@@ Command #2 terminated (pid: 5525, status: 0)
```

Cuestión 2. (5 puntos) Se desea simular el funcionamiento de un centro logístico con capacidad limitada, donde una flota de camiones (todos procesos independientes gestionados con un único semáforo) compite por acceder a los muelles de carga. Una vez un camión consigue un muelle, sus operarios (hilos internos) cargan la mercancía de forma concurrente. El sistema se compone de dos programas ejecutables distintos: `control` y `truck`. Se proporciona el script `test.sh` para probar la solución implementada lanzando 10 camiones contra los muelles. Se pide desarrollar de forma incremental:

Apartado A (1 punto): Programa de inicialización (`control.c`). Este programa es responsable de configurar el entorno de sincronización antes de que lleguen los camiones.

- Debe recibir un argumento entero `N` que indica el número de muelles de carga disponibles.
- Debe crear un semáforo con nombre llamado `/sem_loading_docks`. **Nota:** Asegúrese de que no quedan residuos de la ejecución anterior, p.e., desvinculando el semáforo antes de crearlo.
- Debe inicializar el semáforo con el valor `N`.
- Y mostrar por pantalla: `"System initialized with N loading docks."`, y terminar.

Apartado B (2 puntos): Acceso a muelles (`truck.c`). Se debe implementar una primera versión del programa `truck.c` que gestione el acceso al recinto. Este programa simulará el comportamiento de un único camión. Se ejecutarán múltiples instancias de este programa simultáneamente.

- El programa recibirá dos argumentos enteros: `id` como identificador del camión, y `packages` como número total de paquetes que debe cargar.
- El camión intentará acceder a un muelle utilizando el semáforo `/sem_loading_docks`. Deberá mostrar `"[Truck <id>] Waiting for a loading dock..."` antes de solicitar permiso. Si no hay muelles libres, el proceso se bloqueará hasta que uno se libere. Una vez conseguido el acceso, mostrará `"[Truck <id>] Entering the loading dock."`. No es necesario respetar el orden de llegada, por simplicidad.
- Se simulará la carga simplemente con una llamada a `usleep(1000*packages)`, **sin usar hilos todavía**, simulando que es el propio conductor quien carga todo.
- Después, se mostrará: `"[Truck <id>] Loading finished (Total loaded: <packages>). Exiting."`. Liberará el muelle (semáforo) y terminará.
- Se debe validar este apartado con `test.sh` antes de pasar al siguiente.

Apartado C (2 puntos): Operarios concurrentes. Se debe modificar el programa `truck.c` del apartado B para optimizar el tiempo de carga usando paralelismo.

- Sustituya la espera `usleep` secuencial del apartado anterior por la creación de 2 hilos (operarios).
- Divida `packages` entre los dos hilos.
- Cada operario sumará su parte, **de uno en uno**, a una variable global compartida `current_load` (inicializada a 0), sin que se den condiciones de carrera. Para simular el tiempo de carga, cada operario hará una llamada a `usleep(1000)` (1 milisegundo) en cada iteración de su bucle de carga.
- El hilo principal (`main`) esperará a que terminen los operarios antes de liberar el muelle y salir como en el apartado anterior, pero cambiando el mensaje de salida a `"[Truck <id>] Loading finished (Total loaded: <current_load>). Exiting."`.

```
$ ./control 2
System initialized with 2 loading docks.
$ bash test.sh
[Truck 101] Waiting for a loading dock...
[Truck 102] Waiting for a loading dock...
[Truck 101] Entering the loading dock.
[Truck 102] Entering the loading dock.
[Truck 103] Waiting for a loading dock...
... (pasa el tiempo de carga de 101) ...
[Truck 101] Loading finished (Total loaded: 50). Exiting.
[Truck 103] Entering the loading dock.
...
```